

TailorSync

Presentation, Live Demo & Panel Defense Guide

Updated edition – aligned with the final system, the 13-slide deck and the official Final Presentation Guidelines

Presentation	10 minutes
Live demonstration	12 minutes
Individual Q&A	5 minutes
Total	27 minutes

Team: A.G.S.V. Wimalasiri (CIT-24-01-0110) · W.A.E.M. Wijayarathna (CIT-24-01-0436) · N.D.H.A. Madubhashitha (CIT-24-01-0065) · D.M.J.B. Dissanayake (CIT-24-01-0004) · Manuwendra Rajapaksha (CIT-24-01-0090) · K.P.N.D. Ashokarathna (CIT-24-01-0066)

Faculty of Computing & IT, Sri Lanka Technology Campus · Academic year 2028A

Contents

- 1 Executive summary, timing and the “system story”
- 2 Slide-by-slide speaker plan (10 minutes)
- 3 Member scripts and live runbooks
- 4 12-minute live demo runbook, checklist and backup plan
- 5 Panel defense: likely questions and model answers
- 6 Code walkthrough – what each key file does
- 7 Key numbers to remember
- 8 Corrections from the previous edition – do not say these

1. Executive Summary & Presentation Strategy

TailorSync is a cloud-hosted, AI-assisted management system for small and medium tailoring shops. A Flutter app (Android and web) talks to a FastAPI backend on **Microsoft Azure App Service**, which stores data in **Neon (serverless PostgreSQL)** and uses a **dual-engine AI**: a custom scikit-learn model and an **Azure AI Foundry agent** (`ts-ai-agent` , GPT-4.1-mini). It also offers AI **Virtual Try-On** (Cloudflare Workers AI, FLUX.2), cutting **blueprints**, a self-updating **fabric inventory** and **reports**.

Goal with the panel: prove the system is **real** (deployed, live data), **intelligent** (AI with measured accuracy), **safe** (human-in-the-loop, security, privacy) and **well engineered** (modular code, testing, CI/CD).

1.1 The “system story” every member continues

Chapter	Member	What you prove
The User	K.P.N.D. Ashokarathna	A tailor creates an order in a clean 9-step Flutter wizard.
The Cloud	A.G.S.V. Wimalasiri	The request travels over HTTPS to FastAPI on Azure App Service.
The Brain	W.A.E.M. Wijayarathna	Custom ML or the Foundry agent predicts measurements, fabric and metres.
The Vault	N.D.H.A. Madubhashitha	Neon PostgreSQL stores everything with relational integrity, per business.
The Logic	Manuwendra Rajapaksha	The tailor reviews AI values before saving; orders move through 7 stages.
The Shield	D.M.J.B. Dissanayake	Tests, CI/CD and security keep the deployed system reliable.

Golden rule: say only what the demo can show. Every claim in this guide has been checked against the current code.

2. Slide-by-Slide Speaker Plan (10 minutes)

#	Slide	Speaker	Time	Key line
1	Title	Wimalasiri (lead)	0:30	"We are Group 07 – TailorSync, AI-powered tailoring management."
2	Problem statement	Rajapaksha	0:50	"15–20% of hand-written measurement profiles contain errors."
3	Related work	Rajapaksha	0:50	Read the "Unlike existing systems..." line slowly.
4	Aim, objectives, solution	Wijayarathna	0:50	"Five measurable objectives – we will show each as achieved."
5	Architecture	Wimalasiri	1:00	Narrate left to right: app → API → AI engines / DB / external APIs.
6	Datasets & tools	Madubhashitha	0:50	"10 curated datasets; shirts and trousers train the custom ML."
7	Requirements & features	Ashokarathna	0:50	Four AI features + targets: $\geq 85\% \pm 2$ cm, ≤ 5 s.
8	Results – AI workflow	Wijayarathna	1:00	"90% within ± 2 cm, MAE 1.35 cm, about 2–6 s per AI step."
9	Results – outputs	Ashokarathna	0:40	Estimate → blueprint → Try-On → reports.
10	Testing	Dissanayake	0:50	"35/35 core cases plus 8/8 new-feature tests passed."
11	Contributions	Each member, 1 line	0:40	Name – module – one key contribution.
12	Limitations & further work	Madubhashitha	0:30	Be honest: small datasets, ML covers shirts/trousers only.
13	Live demo & Q&A	Wimalasiri	0:10	Hand over to the demo driver.

Timing tip: rehearse with a stopwatch; total above is 9:50. If you run long, shorten slides 6 and 7 – never the results slides.

3. Member Scripts & Live Runbooks

Member 1 A.G.S.V. Wimalasiri - Backend, Cloud & Azure

Focus: backend architecture, Azure deployment, AI integration plumbing, notifications.

"I built the backend of TailorSync with Python and FastAPI. We chose Python because it is the standard language for machine learning, and FastAPI because it is fast and validates every request automatically with Pydantic. The code follows a router-and-service pattern: each area – orders, customers, AI, inventory, reports, try-on – has its own router, and the business logic lives in services. The backend is deployed on Microsoft Azure App Service and is redeployed automatically from GitHub. It connects to the Azure AI Foundry agent using the App Service's managed identity, so no AI key is stored in the code. When an order is assigned or completed, e-mails are sent on a separate background thread, so the tailor never waits for the mail server."

Live runbook

1. Azure Portal → App Service `tailorsync-api-prod` → **Overview**: show it is running in the cloud.
2. **Log stream**: while Member 3 saves an order, point out the `POST /api/v1/orders/` request arriving.
3. Optional: open `...azurewebsites.net/docs` to show the auto-generated API documentation.
4. IDE: `backend/app/api/v1/routers/orders.py` – show the short route functions and the `Depends(get_current_active_user)` security check.

Member 2 W.A.E.M. Wijayarathna - Dual-Engine AI

Focus: the custom ML model and the Foundry agent, and why we have both.

"TailorSync does not just store measurements – it completes them. I built a dual-engine AI. The first engine is our own machine-learning model: scikit-learn Extra-Trees regressors trained on our curated shirt and trouser datasets and saved with Joblib. For a shirt the tailor enters shoulder, height and chest; the model predicts collar size, sleeve opening and both sleeve lengths. For a trouser, height, waist and seat give knee height, round knee, round end and crotch. We take the prediction of every tree and group them with K-Means, so the tailor sees the main prediction plus alternatives with how strongly the model supports each one. It answers in milliseconds."

The second engine is an Azure AI Foundry agent called `ts-ai-agent`, running GPT-4.1-mini. It supports every garment type and also recommends fabrics with a match percentage and estimates the metres needed. At first the agent took about 30 seconds per step; by switching from GPT-5-mini to GPT-4.1-mini, removing the File Search tool and sending a compact message, we brought it down to about 2-6 seconds. Our measured accuracy is 90% of predictions within ± 2 cm, with a mean absolute error of 1.35 cm."

Live runbook

1. In the app wizard Step 2, show the two choices: **AI Foundry** and **Custom ML Model**.
2. IDE: `backend/app/services/ml_service.py` – show `joblib.load`, the input-range check and the K-Means alternatives.
3. IDE: `backend/app/services/foundry_client.py` – show the compact JSON `agent_message` and the fallback to the base model.
4. Optional terminal: `python test_foundry_agent.py` to show a live agent answer.

Member 3 K.P.N.D. Ashokarathna - Flutter Frontend & UX

Focus: usability, the 9-step wizard, design system and state management.

"The best AI is useless if the tailor cannot use it. I built the Flutter frontend, which runs from one code base on Android and on the web. We created our own design system – brand-indigo colours from the logo, reusable cards, buttons and fields, and a scissor loading animation – so every screen looks consistent. The heart of the app is the 9-step AI Order Wizard: Customer, Prediction Method, Garment, Priority Input, AI Prediction, Preferences, Fabric Recommendation, Estimation and Review & Assign. Measurement fields are generated from the garment's template. We use Riverpod for state management, so when an order is saved, the dashboard, orders and reports refresh automatically."

Live runbook

1. Drive the app (phone or emulator) through the wizard – see Section 4.
2. Point out the live validation: ranges under each measurement field, red messages for missing input.
3. IDE: `lib/features/orders/presentation/screens/new_order_wizard.dart` (step list) and `providers/orders_provider.dart` (Riverpod providers and refresh trigger).

Member 4 N.D.H.A. Madubhashitha - Database & Data Engineering

Focus: the relational schema, data integrity and the datasets.

"All of this needs reliable storage. We use Neon, a serverless PostgreSQL database, through the SQLAlchemy ORM. Customers, orders, measurements, AI predictions, staff assignments and inventory are separate tables linked with foreign keys, and every record belongs to a business, so one shop can never see another shop's data. The ORM builds parameterised SQL, which protects us from SQL injection. For the inventory we keep a transaction table, so every restock or deduction has an audit trail. I also prepared the ten tailoring datasets: we cleaned them, interpolated missing values and standardised fractional inches."

Live runbook

1. Show the ER diagram from the report.
2. Neon console → SQL Editor: `SELECT order_id, order_number, status, prediction_method, created_at FROM orders ORDER BY created_at DESC LIMIT 1;` – the order just created in the demo appears.
3. Optional: `SELECT fabric_name, quantity_m FROM inventory_items ORDER BY fabric_name;` to show stock after the deduction.

Test both queries on the day before the presentation – column names must match your database.

Member 5 Manuwendra Rajapaksha - Order Workflow & Business Logic

Focus: human-in-the-loop, the 7-stage workflow, staff assignment, roles.

"AI can make mistakes, so TailorSync keeps the tailor in control. In Step 5 of the wizard the tailor chooses the recommended prediction or an alternative, and can edit any value before continuing; the fabric amount can also be overridden. Nothing is saved until the tailor taps Save Order. Each order then moves through seven stages: Order Received, Cutting, Sewing, Fitting, Quality Check, Ready and Delivered. The owner assigns each order to a staff member, who gets an e-mail and sees it in their Tasks. Owners and staff have different permissions - inventory, reports and staff management are owner-only."

Live runbook

1. In the wizard Step 5, tap **Edit Measurements** and change one value - the human override.
2. In **Tasks**, tap **Update Stage** on the new order and move it to **Cutting**; the progress bar updates.
3. Optional: log in as a staff account to show the reduced menu (Tasks, Orders, Profile).

Member 6 D.M.J.B. Dissanayake - QA, Testing & DevOps

Focus: testing evidence, CI/CD, reliability and security.

"I was responsible for quality and DevOps. We tested at three levels: unit tests for functions such as password validation and inventory balances; integration tests that call the real API, database and AI services; and end-to-end system tests of the full wizard. We also benchmarked every AI step. All 35 core test cases and all 8 tests for the newer features passed. Our GitHub Actions pipeline installs the dependencies, packages the backend and deploys it to Azure on every push to the main branch, so the live server always runs the latest reviewed code. For security we use bcrypt-hashed passwords, JWT tokens, Google reCAPTCHA on sign-in, HTTPS everywhere and owner-only endpoints."

Live runbook

1. Terminal in `backend`: run one quick test script, e.g. `python test_joblib.py` or `python test_auth_system.py`.
2. GitHub → **Actions**: show the green ticks for recent deployments.
3. Show the test-case table from the report (slide 10).

Do **not** say the pipeline "runs every test and blocks deployment". It currently builds and deploys; tests are run by the team before pushing. If asked, say adding an automated test gate is planned.

4. 12-Minute Live Demo Runbook

Time	Driver	Action and what to say
0:00–1:00	Ashokarathna	Open the app → splash (scissor loader) → Sign in with reCAPTCHA → Home dashboard: “Today at a glance”, New Order and Try-On cards.
1:00–2:00	Ashokarathna	Customers → Add New → fill Full name and phone → Save. Show the friendly validation.
2:00–6:00	Ashokarathna + Wijayarathna	New Order: Step 1 customer → Step 2 AI Foundry → Step 3 Short Trouser → Step 4 Height 25, Waist 32, Seat 34 → Step 5 “Microsoft Foundry is Thinking...” → results and alternatives (Rajapaksha: edit one value) → Step 6 Office/Cold/Modern Fit → Step 7 top fabrics with match % and stock badge → Step 8 metres estimate → Step 9 assign staff → Save Order .
6:00–7:00	Wimalasiri	Azure Log stream showing the POST; then Madubhashitha runs the Neon query to show the saved row.
7:00–8:00	Rajapaksha	Open the order → measurements, fabric, cutting blueprint → Tasks → Update Stage to Cutting.
8:00–9:30	Ashokarathna	Virtual Try-On: photo → Kandyan saree, Royal blue, Silk → consent → Generate.
9:30–11:00	Madubhashitha	Inventory (the fabric was deducted) → Reports: period tabs, line chart, status donut, AI usage split.
11:00–12:00	Wijayarathna	Repeat Step 5 quickly with Custom ML on a shirt to show the instant result and support %.

4.1 Pre-demo checklist (day before and 30 minutes before)

- Push the latest code; wait for the green tick in GitHub Actions; **Restart** the App Service; open the app once to wake the server.
- Log in on the demo phone and on a backup device; keep the owner and one staff account ready.
- Make sure the demo customer exists and at least one fabric is **tracked** in Inventory so the deduction is visible.
- Check the Try-On counter has images left today (limit resets 5:30 AM Sri Lanka time).
- Open tabs in advance: Azure Portal (Log stream), Neon SQL Editor, GitHub Actions, IDE with the files from Section 6.
- Phone: full battery, Do Not Disturb, screen mirroring tested; use a mobile hotspot as backup internet.

4.2 Backup plan

- Record a full screen-capture of the demo flow beforehand; play it if the network or Azure fails.
- If the Foundry step is slow, switch to **Custom ML** on a shirt or trouser and say: “this is exactly why we built two engines.”
- If Try-On is out of images, show a previously generated result from the gallery.

5. Panel Defense - Likely Questions & Model Answers

Q1. How does data flow from the app to the database?

(Member 1 or 4) The tailor taps Save → the Flutter app sends JSON in an HTTPS POST with a JWT token to FastAPI on Azure → the auth dependency checks the user and business → a Pydantic schema validates types (a bad request gets HTTP 422) → the service layer creates SQLAlchemy objects, deducts tracked fabric stock and commits one transaction to Neon PostgreSQL → the API returns the saved order and the app refreshes its Riverpod providers.

Q2. Why Joblib and not plain pickle?

(Member 2) Joblib is built for objects holding large NumPy arrays, such as scikit-learn tree ensembles, so it saves and loads them more efficiently. We load the shirt and trouser models once at server start-up, so each prediction only runs the model.

Q3. What happens if Azure AI Foundry is down?

(Member 1 or 2) Two layers of protection. First, if the agent call fails, `foundry_client.py` automatically falls back to calling the deployed model directly with the relevant dataset rows as context. Second, the Custom ML engine runs inside our own backend and does not depend on Foundry at all, so shirts and trousers can always be predicted.

Q4. Which state management do you use in Flutter, and why?

(Member 3) **Riverpod**. Providers such as `ordersProvider` and `customersProvider` hold data outside the widgets, so we avoid passing data through every screen. After a save we bump a `refreshTriggerProvider` or invalidate a provider, and every screen watching it rebuilds automatically.

Q5. How are e-mails sent without slowing the app?

(Member 1 or 5) `email_service.py` starts a separate background thread for each e-mail (staff invitation, task assignment, order finished). The API responds immediately; the thread connects to the SMTP server on its own, and a failure is logged without affecting the order.

Q6. How did you measure the 90% accuracy?

(Member 2 or 6) We compared predicted values with human-taken measurements on a control set of 35 real customer records and counted predictions within ± 2 cm (90%), and calculated the mean absolute error (1.35 cm).

Q7. Why was the agent slow, and how did you fix it?

(Member 2) GPT-5-mini spent time on reasoning tokens, and File Search added about 4,500 input tokens per call. We switched the agent to GPT-4.1-mini, removed File Search, wrote focused instructions and send a compact JSON message – from about 30 s to about 2–6 s per step.

Q8. How does the inventory stay correct?

(Member 4 or 5) A fabric the AI recommends is added automatically as “not tracked” at 0 m. Once the owner sets its stock it becomes tracked: saving an order deducts the estimated metres and deleting the order restores them. Every change is written to a transaction table with the balance after it.

Q9. How do you protect customer data and photos?

(Member 6) HTTPS, bcrypt-hashed passwords, JWT, reCAPTCHA, business-level data isolation and owner-only endpoints. AI services never receive customer names or contact details. Try-On photos are sent only to generate the image and are not stored on our servers. Our Privacy Policy follows Sri Lanka's PDPA No. 9 of 2022.

Q10. What would you improve next?

(Any member) Train the custom ML for all 10 garment types with consented shop data, add offline prediction, add an automated test gate to CI/CD, customer SMS/WhatsApp updates and a larger field evaluation.

6. Code Walkthrough - What Each Key File Does

File	What to say
<code>backend/app/api/v1/routers/orders.py</code>	Order endpoints (create, list, details, status update, delete). Checks the logged-in user with <code>Depends(get_current_active_user)</code> , validates with Pydantic, hands work to the service layer, triggers task/finished e-mails.
<code>backend/app/services/order_service.py</code>	Creates orders, saves measurements and fabric data, calls the inventory service to deduct or restore stock.
<code>backend/app/services/ml_service.py</code>	Loads the Joblib models, checks inputs are inside the trained ranges, runs the Extra-Trees model, clusters tree outputs with K-Means into alternatives with support %, snaps values to ¼ inch.
<code>backend/app/services/foundry_client.py</code>	Calls the Foundry agent <code>ts-ai-agent</code> through the Responses API with <code>DefaultAzureCredential</code> (managed identity); extracts JSON from the reply; falls back to the base model.
<code>backend/app/services/ai_service.py</code>	Fabric recommendation and estimation; attaches live stock status from the inventory to each recommended fabric.
<code>backend/app/services/inventory_service.py</code> & <code>routers/inventory.py</code>	Auto-creates fabrics, deducts/restores stock, restocks, history, low-stock alerts; owner-only endpoints.
<code>backend/app/api/v1/routers/reports.py</code>	One overview endpoint: KPIs, trend buckets, status, garments, customers, staff and inventory for the chosen period.
<code>backend/app/services/tryon_service.py</code>	Sends the photo and description to Cloudflare Workers AI (FLUX.2 klein) and enforces the daily image limit.
<code>backend/app/schemas/order.py</code> , <code>ml.py</code>	Pydantic models that define the shape of requests and responses (wrong types are rejected with HTTP 422).
<code>backend/app/models/*.py</code>	SQLAlchemy tables and foreign keys: orders, customers, measurements, ai_predictions, staff_assignments, inventory items and transactions.
<code>lib/features/orders/.../new_order_wizard.dart</code>	The 9-step wizard; dynamic measurement fields; calls the AI endpoints; final summary with blueprint.
<code>lib/features/orders/.../providers/orders_provider.dart</code>	Riverpod providers for orders and the refresh trigger used after saves.

<code>lib/features/orders/models/order.dart</code>	<code>fromJson</code> turns the API's JSON into a typed Dart <code>Order</code> object.
<code>lib/ui/</code> (design system)	Shared colours, spacing, cards, buttons, loaders (scissor animation) and page backgrounds used by every screen.
<code>lib/features/profile/.../profile_screen.dart</code>	Reads the user's role (OWNER/STAFF) and shows owner-only options such as Business Profile and Manage Staff.

7. Key Numbers to Remember

Fact	Value
Measurement accuracy (control set of 35)	90% within ± 2 cm · MAE 1.35 cm
Foundry agent response per step	about 2–6 s (was about 30 s)
Custom ML response	milliseconds; shirts and trousers only
Datasets	10 (shirts 40 rows, trousers 27, others 50 each)
Wizard / production	9 steps / 7 stages
Testing	35/35 core + 8/8 new-feature tests passed
User satisfaction	4.33 / 5 (target ≥ 4.0)
Try-On	26 garments, 21 colours, 13 fabrics; daily free limit

8. Corrections from the Previous Edition - Do Not Say These

Previous edition said	Correct statement
Flutter uses Provider and <code>notifyListeners()</code>	We use Riverpod (providers + refresh trigger / invalidate).
Custom ML predicts chest, waist, sleeve from height and weight	Shirt: shoulder, height, chest → collar, sleeve open, short/long sleeve. Trouser: height, waist, seat → knee height, round knee, round end, crotch. Shirts and trousers only.
Orders start in a “Draft” state; a “Confirm AI Measurements” button moves them to Approved/Cutting	No such button. Human-in-the-loop happens in wizard Step 5 (choose/edit) and Step 8 (override metres); orders start at Order Received and move with Update Stage .
E-mails use FastAPI BackgroundTasks	E-mails are sent on a separate background thread (<code>threading.Thread</code> in <code>email_service.py</code>).
Foundry is reached with a raw HTTP prompt in <code>ai_service.py</code>	The agent is called in <code>foundry_client.py</code> through the Azure AI Projects SDK with managed identity, with a fallback model.
Foundry “estimates yardage”	Fabric is estimated in metres .
CI/CD runs every test and deploys only if 100% pass	The pipeline builds and deploys on push to main; tests are run by the team. An automated test gate is future work.
Pydantic in <code>ml.py</code> checks physically possible values	Pydantic checks types; the allowed ranges are checked in <code>ml_service.py</code> using the ranges saved with each model.
Built for iPads / tablets on the shop floor	Built and tested on Android and web; iOS is future work.
“Glassmorphism” UI	Our own design system: brand-indigo theme, frosted floating navigation bar, consistent components.

New features to mention (not in the previous edition): Virtual Try-On with Sri Lankan garments, measurement-annotated cutting blueprints, self-updating fabric inventory, Reports & Analytics with trend charts, Google reCAPTCHA sign-in, and the PDPA-aligned Privacy Policy.